

Using a Network Load Balancer with the NGINX Ingress Controller on Amazon EKS

by Cornell Anthony | on 09 AUG 2019 | in [Amazon Elastic Kubernetes Service](#), [Open Source](#) | [Permalink](#) | [Comments](#) | [Share](#)

[Kubernetes Ingress](#) is an API object that provides a collection of routing rules that govern how external/internal users access [Kubernetes services](#) running in a cluster. An [ingress controller](#) is responsible for reading the ingress resource information and processing it appropriately. As there are different ingress controllers that can do this job, it's important to choose the right one for the type of traffic and load coming into your Kubernetes cluster. In this post, we will discuss how to use an NGINX ingress controller on Amazon EKS, and how to front-face it with a Network Load Balancer (NLB).

What is a Network Load Balancer?

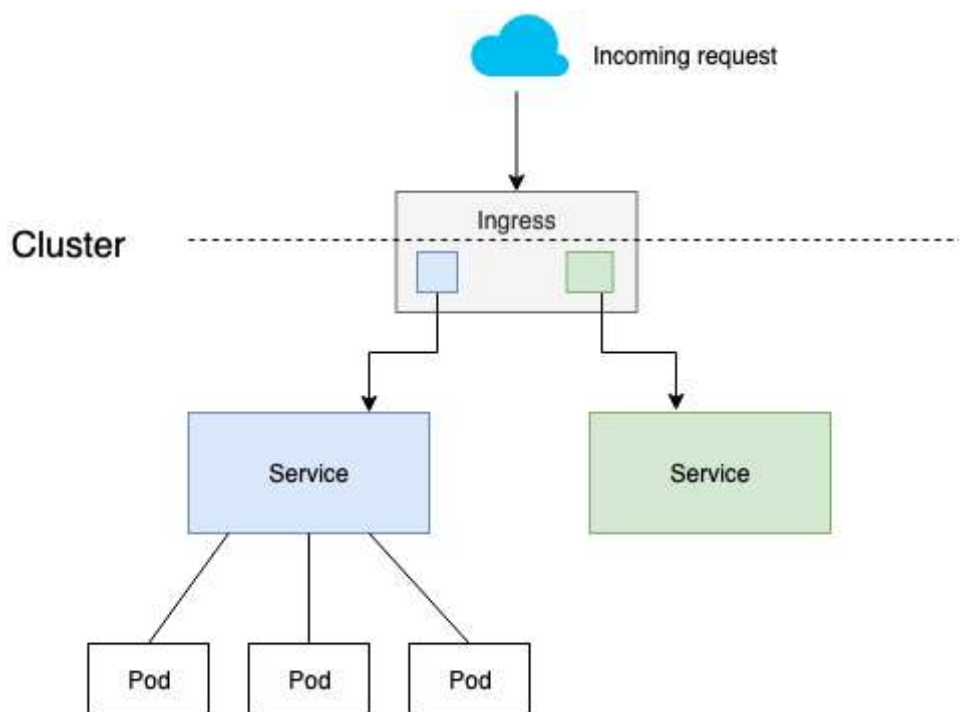
An AWS [Network Load Balancer](#) functions at the fourth layer of the Open Systems Interconnection (OSI) model. It can handle millions of requests per second. After the load balancer receives a connection request, it selects a target from the target group for the default rule. It attempts to open a TCP connection to the selected target on the port specified in the listener configuration.

Exposing your application on Kubernetes

In Kubernetes, there are several different ways to [expose your application](#); using [Ingress](#) to expose your service is one way of doing it. Ingress is not a service type, but it acts as the entry point for your cluster. It lets you consolidate your routing rules into a single resource, as it can expose multiple services under the same IP address.

This post will explain how to use an ingress resource and front it with a NLB (Network Load Balancer), with an example.

Ingress in Kubernetes



Kubernetes supports a high-level abstraction called [Ingress](#), which allows simple host- or URL-based HTTP routing. An Ingress is a core concept (in beta) of Kubernetes. It is always implemented by a third party proxy; these implementations are known as ingress controllers. An ingress controller is responsible for reading the ingress resource information and processing that data accordingly. Different ingress controllers have extended the specification in different ways to support additional use cases.

Typically, your Kubernetes services will impose additional requirements on your ingress. Examples of this include:

- Content-based routing: e.g., routing based on HTTP method, request headers, or other properties of the specific request.
- Resilience: e.g., rate limiting, timeouts.
- Support for multiple protocols: e.g., WebSockets or gRPC.
- Authentication.

An ingress controller is a DaemonSet or Deployment, deployed as a Kubernetes Pod, that watches the endpoint of the API server for updates to the [Ingress resource](#). Its job is to satisfy requests for Ingresses. [NGINX ingress](#) is one such implementation. This blog post implements the ingress controller as a Deployment with the default values. To suit your use case and for more availability, you can use it as a DaemonSet or increase the replica count.

Why would I choose the NGINX ingress controller over the Application Load Balancer (ALB) ingress controller?

The ALB ingress controller is great, but there are certain use cases where the NLB with the NGINX ingress controller will be a better fit. I will discuss scenarios where you would need a NLB over the ALB later in this post, but first let's discuss the ingress controllers.

By default, the NGINX Ingress controller will listen to all the ingress events from all the namespaces and add corresponding directives and rules into the NGINX configuration file. This makes it possible to use a centralized routing file which includes all the ingress rules, hosts, and paths.

With the NGINX Ingress controller you can also have multiple ingress objects for multiple environments or namespaces with the same network load balancer; with the ALB, each ingress object requires a new load balancer.

Furthermore, features like path-based routing can be added to the NLB when used with the NGINX ingress controller.

Why do I need a load balancer in front of an ingress?

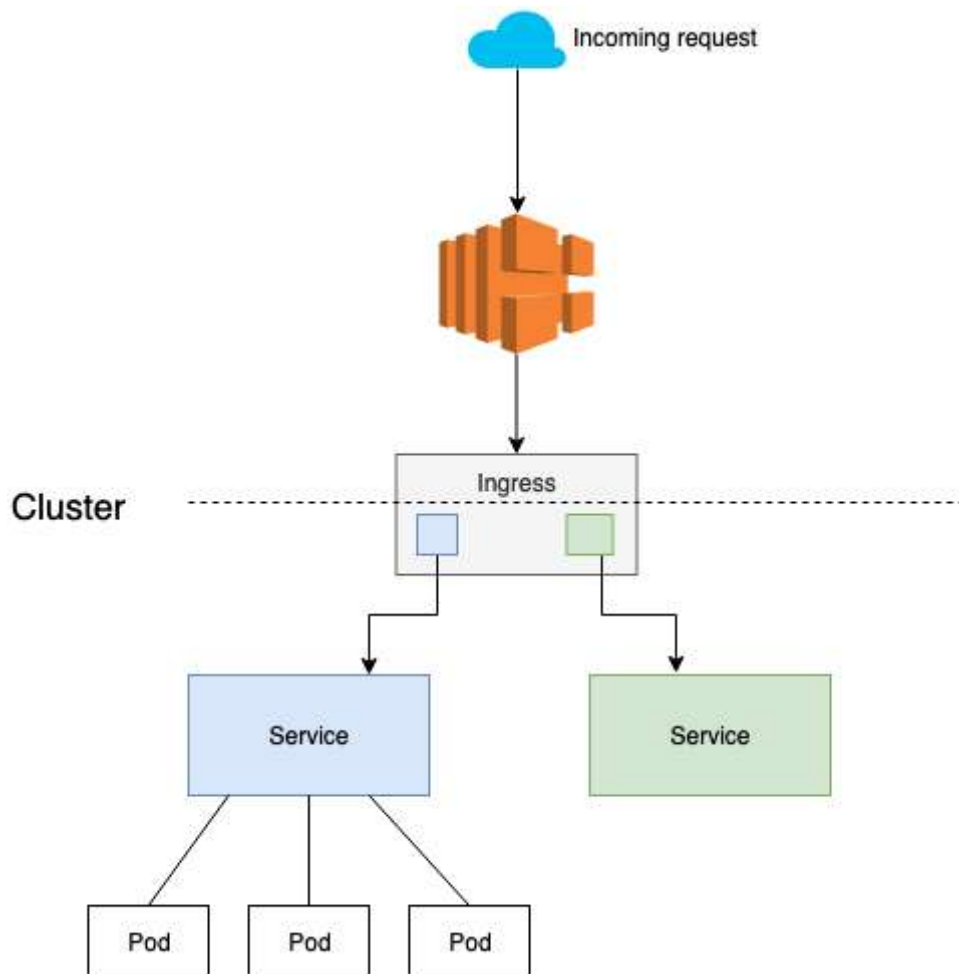
Ingress is tightly integrated into Kubernetes, meaning that your existing workflows around `kubectl` will likely extend nicely to managing ingress. An Ingress controller does not typically eliminate the need for an external load balancer, it simply adds an additional layer of routing and control behind the load balancer.

Pods and nodes are not guaranteed to live for the whole lifetime that the user intends: pods are ephemeral and vulnerable to kill signals from Kubernetes during occasions such as:

- Scaling.
- Memory or CPU saturation.
- Rescheduling for more efficient resource use.
- Downtime due to outside factors.

The load balancer (Kubernetes service) is a construct that stands as a single, fixed-service endpoint for a given set of pods or worker nodes. To take advantage of the previously-discussed benefits of a Network Load Balancer (NLB), we create a Kubernetes service of `type:loadbalancer` with the NLB annotations, and this load balancer sits in front of the ingress controller – which is itself a pod or a set of pods. In AWS, for a set of EC2 compute instances managed by an Autoscaling Group, there should be a load balancer that acts as both a fixed referable address and a load balancing mechanism.

Ingress with load balancer



The diagram above shows a Network Load Balancer in front of the Ingress resource. This load balancer will route traffic to a Kubernetes service (or Ingress) on your cluster that will perform service-specific routing. NLB with the Ingress definition provides the benefits of both a NLB and an Ingress resource.

What advantages does the NLB have over the Application Load Balancer (ALB)?

A Network Load Balancer is capable of handling millions of requests per second while maintaining ultra-low latencies, making it ideal for load balancing TCP traffic. NLB is optimized to handle sudden and volatile traffic patterns while using a single static IP address per Availability Zone. The [benefits](#) of using a NLB are:

- **Static IP/elastic IP addresses:** For each Availability Zone (AZ) you enable on the NLB, you have a network interface. Each load balancer node in the AZ uses this network interface to get a static IP address. You can also use Elastic IP to assign a fixed IP address for each Availability Zone.
- **Scalability:** Ability to handle volatile workloads and scale to millions of requests per second.
- **Zonal isolation:** The Network Load Balancer can be used for application architectures within a Single Zone. Network Load Balancers attempt to route a series of requests from a particular source to targets in a

single AZ while still providing automatic failover should those targets become unavailable.

- **Source/remote address preservation:** With a Network Load Balancer, the original source IP address and source ports for the incoming connections remain unmodified. With Classic and Application load balancers, we had to use HTTP header X-Forwarded-For to get the remote IP address.
- **Long-lived TCP connections:** Network Load Balancer supports long-running TCP connections that can be open for months or years, making it ideal for WebSocket-type applications, IoT, gaming, and messaging applications.
- **Reduced bandwidth usage:** Most applications are bandwidth-bound and should see a cost reduction (for load balancing) of about 25% compared to Application or Classic Load Balancers.
- **SSL termination:** SSL termination will need to happen at the backend, since SSL termination on NLB for Kubernetes is not yet available.

For any NLB usage, the backend security groups control the access to the application (NLB does not have security groups of its own). The worker node security group handles the security for inbound/ outbound traffic.

How to use a Network Load Balancer with the NGINX Ingress resource in Kubernetes

Start by creating the mandatory resources for NGINX Ingress in your cluster:

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/refs/heads/main/deploy/static/provider/cloud/deploy.yaml
```

The above manifest file also launches the Network Load Balancer(NLB).

Now create two services (apple.yaml and banana.yaml) to demonstrate how the Ingress routes our request. We'll run two web applications that each output a slightly different response. Each of the files below has a service definition and a pod definition.

Create the resources:

```
$ kubectl apply -f https://raw.githubusercontent.com/cornellanthony/nlb-ingress-eks,
```

Defining the Ingress resource (with SSL termination) to route traffic to the services created above

If you've purchased and configured a custom domain name for your server, you can use that certificate, otherwise you can still use SSL with a self-signed certificate for development and testing.

In this example, where we are terminating SSL on the backend, we will create a self-signed certificate.

Anytime we reference a TLS secret, we mean a PEM-encoded X.509, RSA (2048) secret. Now generate a self-signed certificate and private key with:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout
tls.key -out tls.crt -subj "/CN=anthonymcornell.com/O=anthonymcornell.com"
```

Then create the secret in the cluster:

```
kubect1 create secret tls tls-secret --key tls.key --cert tls.crt
```

Now declare an Ingress to route requests to `/apple` to the first service, and requests to `/banana` to second service. Check out the Ingress' `rules` field that declares how requests are passed along:

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: example-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "false"
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  tls:
  - hosts:
    - anthonymcornell.com
    secretName: tls-secret
  rules:
  - host: anthonymcornell.com
    http:
      paths:
      - path: /apple
        backend:
          serviceName: apple-service
          servicePort: 5678
      - path: /banana
        backend:
```

```
serviceName: banana-service
servicePort: 5678
```

Create the Ingress in the cluster:

```
kubectl create -f https://raw.githubusercontent.com/cornellanthony/nlb-nginxIngress-eks/n
```

Set up [Route 53](#) to have your domain pointed to the NLB (optional):

```
anthonymcornell.com.      A.
```

```
ALIAS  abf3d14967d6511e9903d12aa583c79b-e3b2965682e9fbde.elb.us-east-1.amazonaws.com
```

Test your application:

```
curl https://anthonymcornell.com/banana -k
```

```
Banana
```

```
curl https://anthonymcornell.com/apple -k
```

```
Apple
```

Can I reuse a NLB with services running in different namespaces? In the same namespace?

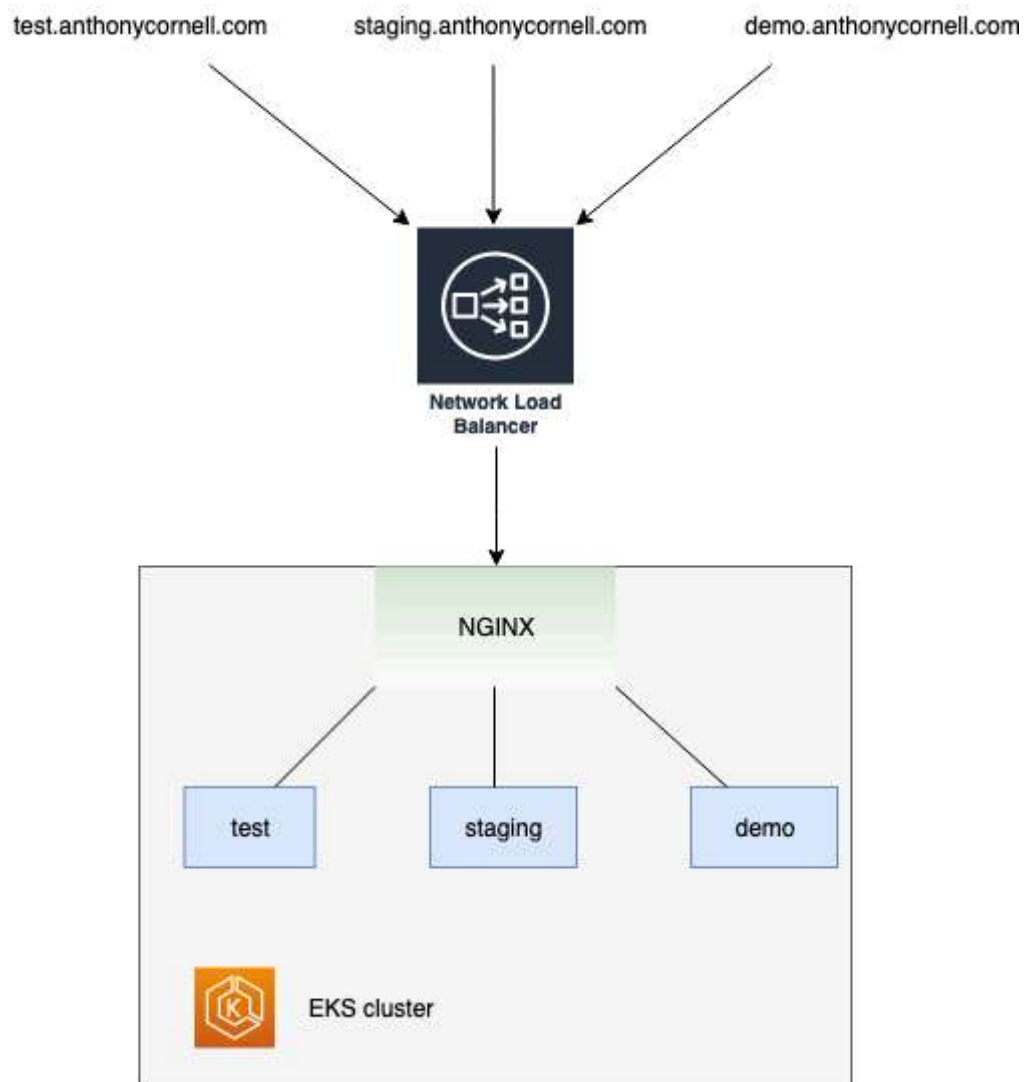
Install the NGINX ingress controller as explained above. In each of your namespaces, define an Ingress Resource.

Example for test:

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: api-ingresse-test
  namespace: test
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  rules:
  - host: test.anthonymcornell.com
    http:
```

```
paths:
- backend:
    serviceName: myApp
    servicePort: 80
  path: /
```

Suppose we have three namespaces – Test, Demo, and Staging. After creating the Ingress resource in each namespace, the NGINX ingress controller will process those resources as shown below:



Cleanup

Delete the Ingress resource:


```
kubect1 delete -f https://raw.githubusercontent.com/cornellanthony/nlb-ingress-eks/r
```

Delete the Secret:

```
kubect1 delete secret tls-secret  
rm tls.crt tls.key
```

Delete the services:

```
kubect1 delete -f https://raw.githubusercontent.com/cornellanthony/nlb-ingress-eks/r
```

```
kubect1 delete -f https://raw.githubusercontent.com/cornellanthony/nlb-ingress-eks/r
```

Delete the NGINX ingress controller:

```
kubect1 delete -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-6
```

We hope this post was useful! Please let us know in the comments.

TAGS: [NGINX](#), [NLB](#)



Cornell Anthony

Cornell is a Cloud Support Engineer at AWS. On a daily basis he provides solutions for customer's cloud architecture in the Networking and DevOps domain. If not at work, you could find him playing soccer, badminton or table tennis .

Like (1)

Share

1 Comment

Log in to comment

Sort by: **Top** ▾

D **Deva** Dec 10, 2024

This is inaccurate - "with the ALB, each ingress object requires a new load balancer."
AWS ALB can share a single ALB for multiple Ingress objects, using annotations like *alb.ingress.kubernetes.io/group.name*, and can group multiple Ingress objects under the same ALB, even across namespaces (if allowed).

👍 Like

💬 Reply

🔗 Share

...